Guía de Trabajos Prácticos: Subconsultas en MySQL 8

Práctica intensiva orientada a resolver problemas mediante consultas anidadas y tablas derivadas, evaluando en cada caso qué dato intermedio se requiere y cómo estructurarlo.

Pauta general: Resolver los problemas formulando subconsultas para calcular o filtrar los datos intermedios necesarios. Evitar el uso de `JOIN` como sustituto de la subconsulta principal cuando el enunciado plantee una condición anidada. No utilizar CTEs (`WITH`).

01 · ESTRUCTURA DE LA BASE DE DATOS: ACADEMIA_ONLINE

categorias
`id_categoria (PK), nombre`

instructores
`id_instructor (PK), nombre, apellido, email, ciudad, sueldo_base`

cursos
`id_curso (PK), titulo, id_categoria (FK), id_instructor (FK), precio, horas`

estudiantes
`id_estudiante (PK), nombre, apellido1, apellido2, ciudad, edad`

inscripciones
`id_inscripcion (PK), id_estudiante (FK), id_curso (FK), fecha_inscripcion, monto_abonado`

02 · SCRIPT DE CREACIÓN DDL (MYSQL 8)

```
CREATE DATABASE IF NOT EXISTS academia_online; USE academia_online; CREATE TABLE categorias ( id_categoria INT PRIMARY KEY, nombre VARCHAR(50) NOT NULL ) ENGINE = InnoDB; CREATE TABLE instructores ( id_instructor INT PRIMARY KEY, nombre VARCHAR(80) NOT NULL, apellido VARCHAR(80) NOT NULL, email VARCHAR(100) UNIQUE NOT NULL, ciudad VARCHAR(80) NOT NULL, sueldo_base DECIMAL(10,2) NOT NULL ) ENGINE = InnoDB; CREATE TABLE cursos ( id_curso INT PRIMARY KEY, titulo VARCHAR(120) NOT NULL, id_categoria INT NOT NULL, id_instructor INT, precio DECIMAL(10,2) NOT NULL, horas INT NOT NULL, FOREIGN KEY (id_categoria) REFERENCES categorias(id_categoria), FOREIGN KEY (id_instructor) REFERENCES instructores(id_instructor) ) ENGINE = InnoDB; CREATE TABLE estudiantes ( id_estudiante INT PRIMARY KEY, nombre VARCHAR(80) NOT NULL, apellido1 VARCHAR(80) NOT NULL, apellido2 VARCHAR(80), ciudad VARCHAR(80) NOT NULL, edad INT NOT NULL ) ENGINE = InnoDB; CREATE TABLE inscripciones ( id_inscripcion INT PRIMARY KEY, id_estudiante INT NOT NULL, id_curso INT NOT NULL, fecha_inscripcion DATE NOT NULL, monto_abonado DECIMAL(10,2) NOT NULL, FOREIGN KEY (id_estudiante) REFERENCES estudiantes(id_estudiante), FOREIGN KEY (id_curso) REFERENCES cursos(id_curso) ) ENGINE = InnoDB;
```

03 · SCRIPT DE DATOS INICIALES (DML)

```
INSERT INTO categorias VALUES (1,'Programacion'),(2,'Diseno'),(3,'Negocios'),(4,'Ciberseguridad'),(5,'Marketing'); INSERT INTO instructores VALUES (1, 'Martin', 'Perez', 'martin.perez@academia.com', 'Mar del Plata', 180000.00), (2, 'Valeria', 'Gomez', 'valeria.gomez@academia.com', 'Balcarce', 240000.00), (3, 'Lucas', 'Diaz', 'lucas.diaz@academia.com', 'Tandil', 195000.00), (4, 'Sofia', 'Romero', 'sofia.romero@academia.com', 'Mar del Plata', 310000.00), (5, 'Ignacio', 'Torres', 'ignacio.torres@academia.com', 'Miramar', 150000.00); INSERT INTO cursos VALUES (101, 'Java Backend & Spring Boot', 1, 1, 45000.00, 60), (102, 'Bases de Datos Relacionales y SQL', 1, 1, 38000.00, 40), (103, 'Figma y UI/UX Design', 2, 2, 28000.00, 30), (104, 'Finanzas Personales e Inversiones', 3, 3, 22000.00, 20), (105, 'Ethical Hacking Inicial', 4, 4, 52000.00, 50), (106, 'Python para Ciencia de Datos', 1, 1, 49000.00, 55), (107, 'Gestion Agil de Proyectos (Scrum)', 3, NULL, 19000.00, 15), (108, 'SEO y Estrategia Digital', 5, NULL, 21000.00, 25); INSERT INTO estudiantes VALUES (1, 'Agustin', 'Suarez', 'Lopez', 'Mar del Plata', 21), (2, 'Camila', 'Vazquez', NULL, 'Balcarce', 27), (3, 'Federico', 'Molina', 'Rios', 'Mar del Plata', 34), (4, 'Julieta', 'Castro', NULL, 'Tandil', 22), (5, 'Gonzalo', 'Herrera', 'Paz', 'Necochea', 41), (6, 'Belen', 'Benitez', 'Gimenez', 'Mar del Plata', 29), (7, 'Marcos', 'Alvarez', NULL, 'Mar del Plata', 19); INSERT INTO inscripciones VALUES (1, 1, 101, '2026-03-01', 45000.00), (2, 1, 102, '2026-03-05', 38000.00), (3, 2, 103, '2026-03-10', 28000.00), (4, 3, 101, '2026-03-12', 45000.00), (5, 3, 105, '2026-03-15', 52000.00), (6, 4, 104, '2026-03-18', 22000.00), (7, 6, 102, '2026-03-20', 38000.00), (8, 6, 106, '2026-03-22', 49000.00);
```

04 · CONSIGNAS DE PRÁCTICA — BLOQUE 1

Ejercicio 01 · Cursos sobre la media

Mostrar el código, título, precio y carga horaria de aquellos cursos cuyo valor supere el precio promedio de todos los cursos de la academia.

Ejercicio 02 · Remuneración máxima docente

Obtener el identificador, nombre, apellido y sueldo del instructor que percibe el sueldo base más alto del cuerpo docente.

Ejercicio 03 · Oferta académica por temáticas

Listar código, título y precio de los cursos correspondientes a las categorías 'Programacion' o 'Ciberseguridad', sin utilizar INNER JOIN para asociar las tablas.

Ejercicio 04 · Cursos sin demanda

Devolver el identificador y título de todos los cursos para los cuales no exista registro de inscripción alguno en el sistema.

Ejercicio 05 · Comparativa de precios contra el área de Negocios

Tomando como conjunto de referencia los precios de los cursos de la categoría 'Negocios': a) Mostrar los cursos cuyo precio sea mayor que al menos uno de esos valores; b) Mostrar los cursos cuyo precio sea mayor que todos esos valores juntos.

Ejercicio 06 · Cursos con alta facturación

Generar un cálculo intermedio que obtenga el total recaudado y la cantidad de inscripciones por curso. A partir de esa información, listar únicamente los cursos que hayan superado los $60.000 de recaudación.

05 · CONSIGNAS DE PRÁCTICA — BLOQUE 2

Ejercicio 07 · Alumnos activos

Mostrar código, nombre y primer apellido de los estudiantes que cuenten con al menos una inscripción registrada en la plataforma.

Ejercicio 08 · Docentes sin asignación actual

Obtener el identificador, nombre y apellido de los instructores que no tengan cursos a su cargo.

Ejercicio 09 · Precios destacados por categoría

Listar código, título, código de categoría y precio de aquellos cursos cuyo valor supere el promedio de precios de los cursos de su propia categoría.

Ejercicio 10 · Conteo docente en la proyección

Listar a todos los instructores mostrando código, nombre, apellido y una columna llamada `total_cursos` con la cantidad de cursos que dicta cada uno (si no dicta ninguno, debe figurar 0).

Ejercicio 11 · Filtrado escalonado

Aislar primero al grupo de instructores con un sueldo base de $180.000 o superior, y sobre ese subconjunto, mostrar únicamente a quienes residan en 'Mar del Plata'.

Ejercicio 12 · Estudiantes en cursos premium

Mostrar código, nombre y apellido de los estudiantes inscriptos en al menos un curso con precio superior a $40.000, e incluir una columna calculada adicional con el total de cursos en los que está inscripto cada estudiante.

06 · CRITERIOS METODOLÓGICOS DE REVISIÓN

- **Forma del resultado:** Identificar previamente si la subconsulta necesaria debe producir un escalar, una lista de valores o un conjunto tabular.
- **Ámbito y correlación:** Determinar si la consulta interna puede resolverse de manera aislada e independiente o si requiere ligar un valor fila por fila desde la consulta contenedora.
- **Cláusulas de ubicación:** Reflexionar si la necesidad del problema corresponde a una condición de selección (`WHERE`), a una columna calculada por fila (`SELECT`) o a un origen de datos intermedio (`FROM`).

Apéndice de referencia para autocorrección conceptual (consultar solo al finalizar):

Ej 01: Escalar en WHERE (comparación simple).
Ej 02: Escalar con MAX() en WHERE.
Ej 03: Subconsulta de lista con IN.
Ej 04: Verificación de ausencia con NOT IN.
Ej 05: Operadores de cuantificación ANY y ALL.
Ej 06: Tabla derivada en FROM con alias.
Ej 07: Comprobación booleana con EXISTS.
Ej 08: Comprobación booleana con NOT EXISTS.
Ej 09: Correlacionada en WHERE (filtro por grupo).
Ej 10: Correlacionada en SELECT (columna agregada).
Ej 11: Subconsulta en FROM (filtro en dos etapas).
Ej 12: Integración doble (WHERE anidado + SELECT).